Reviewer Pack — Minimal p-adic Order of Secant Varieties and Communication C…

Entry 3bd652e0e894 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 17:13:37 UTC

Minimal p-adic Order of Secant Varieties and Communication Complexity of Disjointness

Entry ID: `3bd652e0e894` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-21 17:13:37 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry: Secant Varieties **Field B** (complexity object): Communication Complexity: DISJOINTNESS

Statement:

> ['For any given integer n, there exists a constant C(n) such that the p-adic order of the minimal secant variety of a set of n points in projective space is at least C(n), and this invariant τ_min(sec_n) provides a lower bound on the randomized communication complexity of the disjointness function with τ_min(DISJ_n) = Ω(n).', 'The p-adic order of the secant variety is defined as the smallest integer m such that all coefficients of the defining equations are divisible by p^m for some prime p.', 'This conjecture holds if and only if there exists an algorithm to compute τ_min(sec_n) in polynomial time with respect to n.']

Rationale (proposer's reasoning):

> ['Secant varieties capture the geometric structure of intersection points of curves, which could provide insights into the communication complexity of functions involving these structures.', 'The p-adic order as an invariant may reveal non-trivial algebraic properties that are relevant to the complexity of information exchange between parties.']

Taxonomy category: `COMM_DISJ` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `180997745b769841`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The p-adic order of the minimal secant variety (τ_min(sec_n)) is at least C(n) for all n, where C(n) satisfies τ_min(sec_n) ≥ C(n), and this lower bound is statistically significant with a support fraction >= 0.8 over multiple seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - p-adic order secant varieties AND communication complexity disjointness - algebraic geometry secant varieties OR p-adic order AND communication complexity DISJOINTNESS - minimal p-adic order secant variety algorithm polynomial time AND disjointness function

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
from fractions import Fraction
import math
import sys

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def extended_gcd(a, b):
    if a == 0:
        return b, 0, 1
    gcd, x1, y1 = extended_gcd(b % a, a)
    x = y1 - (b // a) * x1
    y = x1
    return gcd, x, y

def inverse(a, m):
    gcd, x, _ = extended_gcd(a, m)
    if gcd != 1:
        raise ValueError("Inverse does not exist")
    else:
        return x % m

def matrix_mod(A, p):
    return [[a % p for a in row] for row in A]

def matrix_mul(A, B, p):
```

```
    n = len(A)
    m = len(B[0])
    result = [[0] * m for _ in range(n)]
    for i in range(n):
        for j in range(m):
            for k in range(len(B)):
                result[i][j] += A[i][k] * B[k][j]
            result[i][j] %= p
    return result

def matrix_add(A, B, p):
    n = len(A)
    m = len(A[0])
    result = [[0] * m for _ in range(n)]
    for i in range(n):
        for j in range(m):
            result[i][j] = (A[i][j] + B[i][j]) % p
    return result

def matrix_sub(A, B, p):
    n = len(A)
    m = len(A[0])
    result = [[0] * m for _ in range(n)]
    for i in range(n):
        for j in range(m):
            result[i][j] = (A[i][j] - B[i][j]) % p
    return result

def matrix_transpose(A):
    n = len(A)
    m = len(A[0])
    result = [[0] * n for _ in range(m)]
    for i in range(n):
        for j in range(m):
            result[j][i] = A[i][j]
    return result

def gaussian_elimination(A, p):
    n = len(A)
    m = len(A[0])
    rank = 0
    pivot_col = 0
    for i in range(n):
        while pivot_col < m and all(A[j][pivot_col] == 0 for j in range(i, n)):
            pivot_col += 1
        if pivot_col == m:
            break
        A[i], A[min(i + 1, n - 1)] = A[min(i + 1, n - 1)], A[i]
        for j in range(n):
            if i != j:
                factor = (A[j][pivot_col] * inverse(A[i][pivot_col], p)) % p
                A[j] = matrix_sub(A[j], matrix_mul([[factor]], matrix_row(A[i]), p), p)
        pivot_col += 1
        rank += 1
    return rank
```

```
def secant_variety_order(points, p):
    n = len(points)
    m = len(points[0])
    A = [[points[i][j] for j in range(m)] + [1] for i in range(n)]
    A = matrix_mod(A, p)
    return gaussian_elimination(A, p)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    points = [[random.randint(0, p - 1) for _ in range(n)] for _ in range(n)]
    p = random.randint(2, 100)
    try:
        order = secant_variety_order(points, p)
    except Exception as e:
        return {
            "metric_name": "secant_variety_order",
            "metric_value": None,
            "instances_tested": n,
            "conjecture_holds": False,
            "counterexample": str(e)
        }
    return {
        "metric_name": "secant_variety_order",
        "metric_value": order,
        "instances_tested": n,
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(re
    std_value = (sum((r["metric_value"] - mean_value) ** 2 for r in results if r["metric_value"] i
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)
    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='first failing seed' first_failing_seed={first_fa
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_support")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8400f4e3.py", line 132, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8400f4e3.py", line 108, in run_trial
    points = [[random.randint(0, p - 1) for _ in range(n)] for _ in range(n)]
                                  ^
UnboundLocalError: cannot access local variable 'p' where it is not associated with a value
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which prevents us from verifying the conjecture's conditions. | next: Investigate and fix the error in the test code to allow for proper verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14472
2	propose	ollama_remote	glm4:latest	0	0	13922
3	preregistration	ollama_remote	glm4:latest	0	0	9559
4	novelty	ollama_remote	glm4:latest	0	0	8361
5	novelty	ollama_remote	glm4:latest	0	0	9248
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22169
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11968
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13106
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14789
10	judge	ollama_remote	glm4:latest	0	0	11837

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 129430 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/3bd652e0e894.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/3bd652e0e894.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/3bd652e0e894.tar.gz` (if generated)